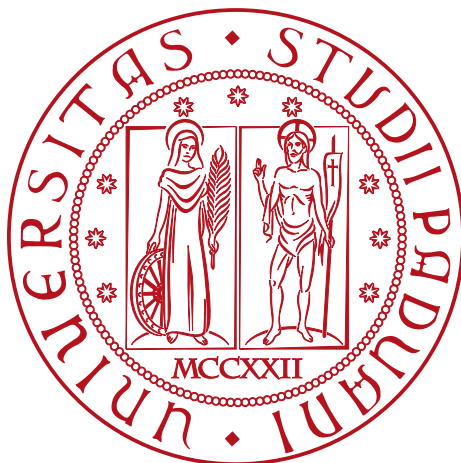


Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA 'TULLIO LEVI-CIVITA'

CORSO DI LAUREA IN INFORMATICA



**Valutazione di pgvector come
database unificato per RAG.**

Tesi di laurea triennale

Relatore

Marco Zanella

Laureando

Davide Lorenzon

Matricola 2101075

I hate perfection. To be perfect is to be unable to improve any further.

— Kurotsuchi Mayuri

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella relatore della mia tesi, per l'aiuto e il continuo sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare con affetto i miei genitori e i miei parenti per il sostegno e per essermi stati vicini durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici per tutti i bellissimi anni passati insieme.

Ci tengo infine a ringraziare i colleghi di Pat SRL e il tutor aziendale Davide Bastianetto per avermi dato l'opportunità di lavorare a questo progetto.

Padova, Settembre 2026

Davide Lorenzon

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di circa trecentoventi ore, dal laureando Davide Lorenzon presso l'azienda Pat SRL. Lo stage è stato condotto sotto la supervisione del tutor aziendale Davide Bastianetto, mentre il prof. Marco Zanella ha ricoperto il ruolo di tutor accademico.

Al centro di questo elaborato vi è la progettazione e lo sviluppo del modulo di retrieval di un sistema *RAG*. Questa architettura è oggi fondamentale poiché permette agli *LLM* di accedere a basi di conoscenza private e informazioni aggiornate senza la necessità di ricorrere a costosi processi di retraining, abbattendo drasticamente il rischio di allucinazioni da parte dell'Intelligenza Artificiale.

Lo scopo principale del progetto è valutare l'adeguatezza, la fattibilità tecnica e le performance dell'estensione *Pgvector* per PostgreSQL, impiegandola come database unificato per l'intera pipeline RAG. Nello specifico, l'obiettivo è verificare se tale tecnologia possa supportare efficacemente l'indicizzazione dei documenti, la ricerca ibrida (combinando ricerca full-text e semantica), l'applicazione di strategie di ranking avanzate e la correlazione relazionale con entità strutturate.

Per convalidare questa ipotesi e fornire una misura rigorosa della qualità del sistema, l'infrastruttura progettata verrà sistematicamente sottoposta a test comparativi contro una pipeline equivalente basata su *Elasticsearch*, tecnologia attualmente adottata all'interno dell'impresa.

L'utilità e il valore aggiunto di questa ricerca risiedono nella potenziale semplificazione dell'infrastruttura IT aziendale. Gestire dati relazionali, testuali e vettoriali all'interno di un unico ecosistema permetterebbe di superare il paradigma della persistenza poliglotta e dei workaround per implementare concetti relazionali come le relazioni tra entità, eliminando la necessità di mantenere sistemi separati. Questo approccio promette di abbattere l'overhead di sincronizzazione dei dati, ridurre i costi di manutenzione sistemistica e garantire transazioni più sicure. Sono previsti test comparativi tra il nuovo sistema e il sistema attualmente in uso.

Organizzazione del testo

- Il **primo capitolo** introduce l'azienda, il progetto e le motivazioni che mi hanno portato a sceglierlo;
- Il **secondo capitolo** descrive l'azienda, il progetto e l'organizzazione del lavoro, definendo gli obiettivi e analizzando i rischi;
- Il **terzo capitolo** descrive l'analisi dei requisiti del progetto, indicando un'analisi degli utenti, i casi d'uso e il tracciamento dei requisiti;
- Il **quarto capitolo** descrive le tecnologie esistenti per risolvere i problemi indicando gli aspetti teorici alla base, gli strumenti che sono stati scelti e con quali criteri;
- Il **quinto capitolo** descrive nel dettaglio le problematiche sorte nel concreto durante lo svolgimento del progetto;
- Il **sesto capitolo** raggruppa le conclusioni tratte dallo svolgimento del progetto.

Convenzioni tipografiche

Durante la stesura del testo ho scelto di adottare le seguenti convenzioni tipografiche:

- Gli acronimi, le abbreviazioni e i termini di uso non comune menzionati vengono definiti nel glossario, situato alla fine del documento (p. 27);
- Per la prima occorrenza dei termini riportati nel glossario viene utilizzata la seguente nomenclatura: *Termine_G*
- I termini in lingua straniera non di uso comune o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*;
- I nomi di funzioni o variabili appartenenti ad un linguaggio di programmazione vengono scritte con un carattere **monospaziato**;
- Le citazioni ad un libro o ad una risorsa presente nella bibliografia (p. 29) saranno affiancate dal rispettivo numero identificativo, es. [1];
- I blocchi di codice sono rappresentati nel seguente modo:


```

1  float Q_rsqr( float number ){
2      long i;
3      float x2, y;
4      const float threehalfs = 1.5F;
5      x2 = number * 0.5F;
6      y = number;
7      i = * (long * ) &y;
8      i = 0x5f3759df - (i>>1);
9      y = * (float * ) &i;
10     y = y * ( threehalfs - ( x2 * y * y ) );
11     //y = y * ( threehalfs - ( x2 * y * y ) );
12     return y;
13 }

```

Codice 1: Codice d'esempio.

Indice

1 Introduzione	1
1.1 L'azienda	1
1.2 Il progetto	1
1.3 Scelta del progetto	3
2 Descrizione stage	5
2.1 Competenze da apprendere	5
2.2 Vincoli	5
2.3 Pianificazione	6
2.4 Analisi dei rischi	6
3 Analisi dei requisiti	15
3.1 Analisi degli Utenti	15
3.1.1 Attori Primari	15
3.1.2 Attori Secondari	16
3.2 Casi d'uso	16
3.2.1 Struttura della scheda descrittiva	16
3.3 Lista delle user stories	17
Epic 1. Gestione utenti	17
US1 Login	17
US2 Registrazione	18
3.4 Tracciamento dei requisiti	18
4 Introduzione Teorica	21
4.1 Aspetti Teorici	21
4.2 Tecnologie	21
4.3 Strumenti	21
5 Descrizione del Lavoro Svolto	23
5.1 TODO	23
6 Conclusioni	25
6.1 Consuntivo finale	25
6.2 Raggiungimento degli obiettivi	25
6.3 Requisiti soddisfatti	25
6.4 Rischi occorsi e mitigati	26
6.5 Valutazione personale	26
Glossario	27
Bibliografia	29

Elenco delle Figure

Figura 1 Logo Pat SRL	1
-----------------------------	---

Elenco delle Tabelle

Tabella 1	Tracciamento dei requisiti funzionali.	19
Tabella 2	Tracciamento dei requisiti di qualità.	19
Tabella 3	Tracciamento dei requisiti di vincolo.	19
Tabella 4	Riepilogo dei requisiti.	19
Tabella 5	Consuntivo orario finale.	25
Tabella 6	Rischi occorsi con la loro mitigazione.	26

Elenco dei Codici Sorgente

Codice 1	Codice d'esempio.	7
----------	------------------------	---

Capitolo 1

Introduzione

In questo capitolo descrivo l'azienda, introduco il progetto, e spiego le motivazioni che mi hanno portato a sceglierlo.

1.1 L'azienda

Pat SRL è un'azienda parte del Gruppo Zucchetti, vanta un'esperienza ultra trentennale nello sviluppo di soluzioni software destinate sia ad aziende private che a istituzioni pubbliche. L'azienda si posiziona come partner tecnologico specializzato nella progettazione di piattaforme per la gestione e l'automazione dei processi aziendali e dei servizi di assistenza e supporto.



Figura 1: Logo Pat SRL

1.2 Il progetto

Il progetto ha come obiettivo principale la riprogettazione e la sostituzione dell'attuale architettura dati utilizzata per la persistenza e il recupero delle informazioni all'interno dei prodotti aziendali.

Attualmente, l'impresa adotta una soluzione consolidata basata sul paradigma della persistenza poliglotta.

1 Introduzione

Tale architettura prevede l'utilizzo congiunto di due sistemi separati: PostgreSQL per la gestione dei dati strettamente relazionali ed Elasticsearch per l'indicizzazione dei documenti, la ricerca full-text, la gestione degli embedding vettoriali e le operazioni di filtraggio avanzato.

Sebbene questo approccio ibrido sia funzionale e ampiamente utilizzato, la divisione dei carichi di lavoro su motori di database differenti comporta diverse limitazioni architettoniche e operative:

- Denormalizzazione dei dati: la natura orientata ai documenti e non relazionale di Elasticsearch obbliga a replicare e denormalizzare i dati relazionali per consentire un filtraggio efficiente, aumentando la ridondanza e forzando a implementare in modo non ottimale funzioni come i join.
- Overhead di sincronizzazione: il mantenimento della coerenza tra il database primario PostgreSQL e il motore di ricerca Elasticsearch richiede complesse pipeline di allineamento, esponendo il sistema a ritardi di sincronizzazione o disallineamenti.
- Mancanza di garanzie ACID globali: operando su sistemi separati, risulta complesso garantire l'atomicità e la consistenza transazionale durante l'inserimento o l'aggiornamento simultaneo di dati strutturati e vettoriali.

Per superare queste criticità, il progetto esplora la transizione verso un paradigma a database unificato. L'obiettivo è accentrare l'intero carico di lavoro su PostgreSQL sfruttando pgvector, un'estensione open-source che introduce il supporto nativo alla persistenza dei vettori di embedding e alle operazioni di algebra lineare direttamente all'interno dell'ecosistema relazionale.

Nello specifico, il nuovo sistema dovrà soddisfare i seguenti requisiti implementativi:

- Ingestion: sviluppo di un modulo dedicato all'inserimento simultaneo di metadati relazionali e documenti non strutturati.
- Ottimizzazione degli indici: sfruttamento delle capacità di indicizzazione full-text native di PostgreSQL e creazione di indici vettoriali dedicati tramite pgvector per garantire l'efficienza scalabile della ricerca semantica.
- Integrazione relazionale: utilizzo di costrutti SQL per correlare dinamicamente i documenti e i vettori alle entità strutturate di dominio preesistenti nel gestionale.
- Ricerca Ibrida: implementazione di una logica di recupero che combini la precisione lessicale della ricerca testuale con la profondità concettuale della ricerca semantica, fondendo i risultati tramite l'algoritmo di Reciprocal Rank Fusion.

1 Introduzione

Al fine di validare rigorosamente l'efficacia di questa nuova architettura unificata, il sistema verrà sottoposto a una fase di benchmarking contro la soluzione attualmente in uso.

I test comparativi si concentreranno sulle seguenti metriche chiave:

- Latenza di interrogazione: misurazione dei tempi di risposta durante la ricerca testuale, semantica e ibrida.
- Velocità di indicizzazione: tempi necessari per l'elaborazione e l'inserimento a database di nuovi record complessi.
- Impatto sullo storage: analisi dello spazio su disco occupato dai dati e dai relativi indici.
- Complessità della pipeline: valutazione qualitativa della semplificazione architetturale.

1.3 Scelta del progetto

Ho scelto questo progetto per 3 ragioni principali:

1. Rilevanza dell'argomento: nell'informatica moderna, l'integrazione di funzionalità basate sull'Intelligenza Artificiale rappresenta un fattore competitivo e innovativo.
2. Centralità della manutenzione: il progetto permette di intervenire direttamente sulla fase più estesa, costosa e critica del ciclo di vita di un prodotto software aziendale.
3. Stack tecnologico all'avanguardia: offre l'opportunità di operare sul campo con strumenti e framework moderni.

1 Introduzione

Capitolo 2

Descrizione stage

In questo capitolo approfondisco l'organizzazione dello stage, il rapporto con l'azienda e svolgo l'analisi dei rischi.

2.1 Competenze da apprendere

Il tirocinio è strutturato per favorire lo sviluppo di un ventaglio di competenze trasversali, che spaziano dalla progettazione architettuale di alto livello fino all'implementazione tecnica.

Dal punto di vista metodologico, lo stage mira a consolidare le capacità di analisi dei requisiti, astrazione dei problemi complessi e progettazione di soluzioni algoritmiche generali e scalabili, promuovendo inoltre la collaborazione attiva con i tutor e gli stakeholder aziendali.

Sotto il profilo tecnico, il percorso formativo permetterà di acquisire e approfondire le seguenti tematiche:

- Sistemi AI e NLP: Comprensione dei fondamenti del Natural Language Processing e integrazione di modelli linguistici e framework associati.
- Database Avanzati: Padronanza nell'utilizzo di PostgreSQL non solo come database relazionale, ma come motore di Information Retrieval vettoriale tramite l'estensione pgvector.
- Progettazione della Ricerca Ibrida: Studio e valutazione delle tecnologie di indicizzazione. Per garantire un confronto equo e rigoroso con Elasticsearch, oltre alla ricerca full-text nativa di PostgreSQL, verrà esplorata l'integrazione di ParadeDB, una soluzione basata su Postgres che promette capacità di ricerca lessicale altrettanto evolute.
- Testing Comparativo: Acquisizione di metodologie per la conduzione di benchmark, definendo metriche di valutazione per misurare le performance e l'efficienza dei sistemi sviluppati.

2.2 Vincoli

Il progetto è soggetto a specifici vincoli architettureali volti a garantire la coerenza con gli obiettivi della ricerca. Il vincolo primario è l'adozione

2 Descrizione stage

esclusiva dell'estensione pgvector su ecosistema PostgreSQL per la gestione e l'interrogazione dei vettori di embedding.

2.3 Pianificazione

Lo stage si articola in 320 ore distribuite su otto settimane da 40 ore.

La pianificazione, derivata dal piano di lavoro, è la seguente:

1. Prima Settimana - Studio e analisi iniziale del nuovo e del vecchio stack tecnologico e setup (40 ore): studio delle tecnologie, setup dell'ambiente di lavoro locale e prove generiche;
2. Seconda Settimana - Analisi comparativa dettagliata di elastic search e Postgres con tentativo di modellazione di un sottoinsieme limitato del problema;
3. Terza Settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale
4. Quarta settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale e ottimizzazione tramite indici
5. Quinta settimana - Strutturazione del modulo di ingestion
6. Sesta settimana - Strutturazione del modulo API e integrazione dei framework di observability
7. Settima settimana - Testing e ottimizzazioni
8. Ottava settimana - Testing e ottimizzazioni

2.4 Analisi dei rischi

I rischi identificati per questo progetto sono classificati con un codice progressivo della forma **RN**, dove **N** è un numero intero incrementale che parte da 01, e decorati con una probabilità di occorrenza, un impatto e una strategia di mitigazione.

Ogni rischio è stato analizzato tenendo conto della complessità di comprendere i casi d'uso del sistema di paragone Elastic, delle sue scelte implementative dovute allo stack tecnologico e dalla comprensione degli interessi sperimentativi dell'impresa.

Codice : R01

Incompletezza o ambiguità dei requisiti

- **Descrizione:**

I requisiti descritti nel piano di lavoro possono risultare incompleti o ambigui, portando a implementazioni non coerenti con le aspettative aziendali.

- **Mitigazione:**

I requisiti vengono analizzati e chiariti prima delle attività di codifica. Viene data priorità ad attività di studio teorico dello stack tecnologico e alla realizzazione di prototipi a diversi livelli di complessità. Sono previsti incontri iterativi con il tutor per definire chiaramente i confini del progetto e le interfacce di comunicazione.

- **Probabilità:** Medio-Alta

- **Impatto:** Medio

Codice : R02

Sovradimensionamento del progetto rispetto alle tempistiche

- **Descrizione:**

Le attività previste potrebbero rivelarsi eccessive rispetto al monte ore disponibile e alle tempistiche di apprendimento, con il rischio di non completare tutti gli obiettivi prefissati.

- **Mitigazione:**

Le attività sono suddivise per priorità (requisiti Obbligatori, Desiderabili, Opzionali). In caso di ritardi o imprevisti strutturali, solo le attività a priorità inferiore e non vincolanti per il core del progetto subiranno ridimensionamenti.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R03

Difficoltà nel coordinamento interno

- **Descrizione:**

La comunicazione con il tutor aziendale o con gli stakeholder potrebbe risultare discontinua, rallentando il processo decisionale tecnico e causando incomprensioni.

- **Mitigazione:**

Vengono pianificati incontri periodici di allineamento, accompagnati da aggiornamenti asincroni frequenti sullo stato di avanzamento. Eventuali blocchi tecnici (blocker) vengono segnalati tempestivamente senza attendere la riunione successiva.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R04

Curva di apprendimento delle tecnologie

- **Descrizione:**

L'assimilazione delle tecnologie necessarie (es. pgvector, framework NLP) potrebbe richiedere un tempo di studio superiore alle stime iniziali.

- **Mitigazione:**

Le prime settimane dello stage includono ore dedicate esplicitamente allo studio della documentazione ufficiale, alla schematizzazione delle architetture e alla realizzazione di proof-of-concept (PoC) isolati.

- **Probabilità:** Bassa

- **Impatto:** Medio

Codice : R05

Incompatibilità o difficoltà di integrazione con il sistema legacy

- **Descrizione:**

Il modulo realizzato dovrà essere testato in un ambiente paragonabile a quello di produzione; potrebbero emergere attriti o incompatibilità nell'interfacciamento con il sistema esistente.

- **Mitigazione:**

Confronto continuo con il team di sviluppo per comprendere a fondo i contratti delle interfacce. Adozione del design pattern «Adapter» fin dalle prime fasi di progettazione per disaccoppiare la logica del nuovo sistema da quella preesistente.

- **Probabilità:** Media

- **Impatto:** Medio

Codice : R06

Saturazione delle risorse durante i benchmark

- **Descrizione:**

L'esecuzione dei test comparativi su volumi di dati enterprise (fino a 500.000 record) potrebbe causare la saturazione delle risorse hardware dell'ambiente di test, falsando le metriche o causando crash di sistema.

- **Mitigazione:**

I test verranno eseguiti in modo incrementale su dataset di dimensioni crescenti.

Verrà implementato un monitoraggio attivo delle risorse tramite il framework di observability per individuare eventuali memory leak o configurazioni sub-ottimali degli indici vettoriali prima dei test massivi.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R07

Rappresentatività dei dati di test e accesso limitato alla produzione

- **Descrizione:**

L'utilizzo di dati generati appositamente per le fasi di test locali, reso necessario dai vincoli di privacy aziendale, potrebbe non riflettere a pieno la reale complessità e varianza del dominio. Questa discrepanza rischia di alterare la valutazione dell'accuratezza degli embedding e di non far emergere casistiche limite verosimili, portando a possibili divergenze di performance tra l'ambiente di sviluppo e le aspettative finali.

- **Mitigazione:**

La validazione seguirà un approccio a due fasi: l'architettura dovrà innanzitutto superare i benchmark di riferimento in ambiente locale utilizzando dei dati di test.

A valle di questo consolidamento, le misurazioni definitive verranno eseguite sui dati reali operando esclusivamente all'interno dei server proprietari aziendali, nel pieno rispetto delle direttive di sicurezza.

- **Probabilità:** Alta

- **Impatto:** Medio

Codice : R08

Difficoltà nella valutazione oggettiva dei risultati di Retrieval

- **Descrizione:**

L'assenza di metriche standardizzate o un'errata interpretazione dei risultati potrebbe portare a conclusioni soggettive, rendendo impossibile valutare se il nuovo sistema eguaglia o supera la soluzione precedente.

- **Mitigazione:**

Le metriche di valutazione quantitativa verranno definite esplicitamente nella fase di analisi dei requisiti.

Queste corrispondono alle metriche attualmente usate per la valutazione del sistema attuale

È inoltre previsto un caso d'uso specifico per la produzione di una dashboard di monitoraggio, con criteri di accettazione e soglie minime di qualità chiaramente prestabiliti.

- **Probabilità:** Media

- **Impatto:** Alto

Codice : R09

Sbilanciamento metodologico nel confronto

- **Descrizione:**

Elasticsearch è un motore di ricerca nativo con analizzatori linguistici avanzati, mentre PostgreSQL nasce come RDBMS. Un set di test limitato rischierebbe di favorire asimmetricamente una tecnologia rispetto all'altra, invalidando l'equità del benchmark.

- **Mitigazione:**

Il set di query di test verrà progettato per essere eterogeneo e rappresentativo dei reali casi d'uso aziendali. Includerà intenzionalmente sia scenari che valorizzano PostgreSQL, in particolare i join, sia scenari che stressano le capacità native di Elasticsearch, funzionalità di querying avanzate come phrase queries e analisi linguistica complessa.

- **Probabilità:** Alta

- **Impatto:** Alto

Codice : R10

Dispersione del perimetro di test su tecnologie alternative

- **Descrizione:**

L'evoluzione rapida del panorama RAG solleva l'interrogativo su alternative tecnologiche, come Open-Search.

La loro valutazione va fuori dagli interessi dell'impresa per questo specifico tirocinio e rischia di aggiungere attività di studio non utili alla realizzazione del progetto.

- **Mitigazione:**

I confini del tirocinio sono rigidamente circoscritti al confronto diretto tra la soluzione in uso, basata su Elasticsearch, e le tecnologie che l'impresa vuole valutare, PostgreSQL + pgvector.

Eventuali tecnologie alternative verranno affrontate esclusivamente a livello teorico nel capitolo di analisi dello «Stato dell'Arte».

- **Probabilità:** Bassa

- **Impatto:** Medio

Codice : R11

Sovrapposizione tra le performance di Retrieval e Generation

- **Descrizione:**

L'architettura RAG si compone di due fasi: la fase di retrieval che si occupa del recupero di informazioni e la generazione della risposta. Nel sistema attuale solo la parte di retrieval e ingestion di documenti è collegata strettamente a Elasticsearch, le altre parti del sistema sono gestite in python puro.

- **Mitigazione:**

Vengono posti dei rigidi confini di sistema. L'implementazione, l'analisi e il testing si concentreranno unicamente sulla componente di Retriever e sul relativo modulo di ingestion. La valutazione ignorerà la qualità dell'output generativo dell'LLM, misurando esclusivamente la pertinenza e la velocità dei documenti recuperati.

- **Probabilità:** Bassa

- **Impatto:** Alto

2 Descrizione stage

Capitolo 3

Analisi dei requisiti

In questo capitolo effettuo l'analisi degli utenti, sviluppo le user stories e compongo la lista dei requisiti dividendoli per tipologia e necessità.

3.1 Analisi degli Utenti

Al fine di modellare correttamente le interazioni e definire i confini del sistema oggetto del tirocinio, è necessario individuare gli attori coinvolti. In accordo con le metodologie di ingegneria del software, gli attori comprendono sia gli utenti umani sia i sistemi software esterni che interagiscono direttamente con l'architettura.

Gli attori sono classificati in due categorie:

- Attori Primari: entità che avviano i casi d'uso e richiedono un servizio al sistema.
- Attori Secondari: servizi o sistemi esterni invocati dal sistema per completare una specifica elaborazione.

3.1.1 Attori Primari

- **Companion**

Con il termine **Companion** si identifica l'applicativo aziendale di Intelligenza Artificiale di livello superiore. Questo attore software funge da client principale: è il sistema reale all'interno del quale il modulo sviluppato durante lo stage verrebbe integrato (a scopo esplorativo).

Il Companion interagisce con il sistema per due scopi fondamentali: delegare l'ingestione dei dati documentali e interrogare la base di conoscenza tramite la pipeline RAG per ottenere il contesto necessario alla generazione delle risposte.

3 Analisi dei requisiti

- **Supervisore**

Rappresenta l'utente tecnico qualificato. Il suo ruolo è quello di interfacciarsi con il sistema a scopo di analisi, validazione e benchmarking. Il Supervisore avvia i test prestazionali, monitora l'infrastruttura e raccoglie le metriche necessarie per valutare e comparare le prestazioni del nuovo database unificato rispetto alla soluzione correntemente usata in produzione.

3.1.2 Attori Secondari

- **Modello di Embedding**

Si tratta di un attore software esterno che fornisce il servizio di vettorizzazione. Il sistema oggetto dello stage invoca questo attore delegandogli il compito computazionale di trasformare i chunk di testo grezzo in vettori numerici, questi vettori verranno poi usati per popolare l'indice vettoriale e per l'esecuzione della ricerca semantica.

3.2 Casi d'uso

In questa sezione vengono definiti i casi d'uso del sistema. Ogni caso d'uso è univocamente identificato da una sigla progressiva (es. **UC-X**) ed è corredato da una scheda descrittiva analitica. Gli attori menzionati fanno diretto riferimento alle definizioni riportate nella sezione Capitolo 3.1.

3.2.1 Struttura della scheda descrittiva

Ciascun caso d'uso è documentato attraverso le informazioni elencate nella tabella seguente. Al fine di mantenere la documentazione concisa, i campi non rilevanti o non applicabili a uno specifico scenario verranno omessi.

Campo	Descrizione
Grafico UML	Rappresentazione visiva dello scenario tramite diagramma UML dei casi d'uso.
Attore principale	Entità esterna che avvia l'interazione con il sistema per raggiungere un obiettivo specifico.
Attore secondario	Sistema o servizio esterno invocato dal sistema per completare una determinata funzionalità.

3 Analisi dei requisiti

Campo	Descrizione
Scenario principale	La sequenza nominale di operazioni necessaria per portare a compimento il caso d'uso senza interruzioni.
Precondizioni	Stato del sistema o requisiti che devono essere necessariamente soddisfatti prima dell'avvio del caso d'uso.
Postcondizioni	Stato del sistema o modifiche persistenti che si verificano al termine della corretta esecuzione dello scenario principale.
Scenario alternativo	Flussi di esecuzione secondari che deviano dallo scenario base, tipicamente per gestire anomalie, errori o percorsi non standard.
Inclusioni	Relazione verso un altro caso d'uso, il cui comportamento è obbligatoriamente e incondizionatamente incorporato nello scenario corrente.
Estensioni	Relazione che introduce un comportamento opzionale o alternativo, attivato unicamente al verificarsi di specifiche condizioni.
Specializzazioni	Varianti strutturali del caso d'uso generale. I casi d'uso specializzati sono tra loro mutualmente esclusivi ed ereditano i comportamenti dello scenario base.
Trigger	L'evento, l'azione o la condizione scatenante che innesci l'esecuzione del caso d'uso.

3.3 Lista delle user stories

Epic 1. Gestione utenti

US1 Login

Descrizione: Come utente non autenticato, voglio poter fare il login per accedere alle funzionalità dell'applicativo.

Acceptance criteria:

3 Analisi dei requisiti

1. L'utente deve poter inserire le proprie credenziali (email e password) in un modulo di login.
2. Se l'utente inserisce credenziali corrette, il sistema reindirizza l'utente alla dashboard.
3. Se l'utente inserisce credenziali errate, il sistema mostra un messaggio di errore.

US2 Registrazione

Descrizione: Come utente non autenticato, voglio potermi registrare per accedere alla piattaforma.

Acceptance criteria:

1. L'utente deve poter compilare un modulo di registrazione con i campi richiesti (nome, cognome, email, password).
2. Il sistema effettua la verifica che l'utente non sia già registrato in piattaforma.
3. Dopo la verifica, il sistema invia una email all'utente che si sta registrando con un codice per poter attivare il proprio account.
4. Quando l'utente inserisce il codice ricevuto nella webapp, il suo account viene attivato e ha la possibilità di accedere.

3.4 Tracciamento dei requisiti

Ad ogni requisito è associato un codice costruito in base alle sue caratteristiche:

(F/Q/C)(M/D/O)R

F (*Functional*): definisce una funzione di un sistema o dei suoi componenti;

Q (*Qualitative*): rappresentano come il sistema deve essere per soddisfare i requisiti dello stakeholder;

C (*Constraint*): rappresentano dei vincoli o dei limiti che il sistema deve rispettare;

M (*Mandatory*): irrinunciabili per qualcuno degli stakeholder;

D (*Desirable*): non strettamente necessari ma a valore aggiunto riconoscibile;

O (*Optional*): relativamente utili oppure contrattabili anche in fasi avanzate del progetto;

R (*Requirement*): requisito

In Tabella 1, Tabella 2 e Tabella 3 sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

3 Analisi dei requisiti

Codice	Descrizione	Fonti
--------	-------------	-------

Tabella 1: Tracciamento dei requisiti funzionali.

Codice	Descrizione	Fonti
--------	-------------	-------

Tabella 2: Tracciamento dei requisiti di qualità.

Codice	Descrizione	Fonti
--------	-------------	-------

Tabella 3: Tracciamento dei requisiti di vincolo.

Di seguito, nella Tabella 4 ho inserito il riepilogo dei requisiti, suddivisi per tipologia e necessità.

Tipo	Mandatory	Desirable	Optional	Somma
Functional	Totale			

Tabella 4: Riepilogo dei requisiti.

3 Analisi dei requisiti

Capitolo 4

Introduzione Teorica

In questo capitolo approfondisco le basi teoriche rilevanti per la realizzazione del progetto, le tecnologie rilevanti per il progetto e relativi ruoli nella risoluzione dei problemi affrontati, quali strumenti sono stati adottati e altri strumenti adottati durante lo sviluppo.

4.1 Aspetti Teorici

4.2 Tecnologie

4.3 Strumenti

4 Introduzione Teorica

Capitolo 5

Descrizione del Lavoro Svolto

In questo capitolo approfondisco le problematiche affrontate nel concreto e le relative soluzioni.

5.1 TODO

Definire la struttura del capitolo.

5 Descrizione del Lavoro Svolto

Capitolo 6

Conclusioni

In questo capitolo traggio le conclusioni sul progetto.

6.1 Consuntivo finale

Una volta terminato il progetto ho redatto il consuntivo orario finale nella Tabella 5 che suddivide in maniera approssimata le ore dedicate alle varie fasi.

Fase	Ore
<i>Onboarding</i> del progetto	5
Analisi dei requisiti	30
...	...
Totale	320

Tabella 5: Consuntivo orario finale.

6.2 Raggiungimento degli obiettivi

6.3 Requisiti soddisfatti

Arrivato alla fine del progetto ho implementato...

)

6 Conclusioni

6.4 Rischi occorsi e mitigati

I rischi emersi durante lo stage sono riportati in Tabella 6.

Descrizione	Mitigazione
R1 – Descrizione del rischio	Soluzione

Tabella 6: Rischi occorsi con la loro mitigazione.

6.5 Valutazione personale

Glossario

Elasticsearch: Motore di analisi e ricerca distribuita. Funge da piattaforma di retrieval e salva dati strutturati, non strutturati e dati vettoriali. 5

LLM – Large Language Model: Modello di intelligenza artificiale addestrato su enormi quantità di testo per comprendere e generare linguaggio naturale, come GPT, Gemini o Mistral. Viene impiegato in compiti di analisi, estrazione di informazioni e generazione testuale. 5

Pgvector: Estensione per PostgreSQL, un database management system, che semplifica l'utilizzo dei vettori, consentendoti di archivarli, cercarli e indicizzarli direttamente nel tuo database relazionale. 5

RAG – Retrieval Augmented generation: Tecnica che permette ai LLM di reperire e incorporare informazioni da fonti di dati esterne.

Questo permette di recuperare informazioni rilevanti da database, documenti caricati, fonti web, ecc...

Il vantaggio principale è l'attualità delle risposte fornite dall'LLM e la possibilità di non dover usare documenti privati in fase di addestramento. 5

Bibliografia